In [1]:
```
#Assignment-3
#PRN- 260240128008           PRN-260240128036
#Name-Ashlesha Tayade        Name-Ruchika Gaikwad
```

In [2]:
```
import numpy as np
```

In [3]:
```
#Set1
#Create a 3×3 array of all True Boolean values
arr_b = np.full(9,True).reshape(3,3)
arr_b
```

Out[3]:
```
array([[ True,  True,  True],
       [ True,  True,  True],
       [ True,  True,  True]])
```

In [4]:
```
arr = np.arange(3, 12).reshape(3,3)
arr
```

Out[4]:
```
array([[ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

In [5]:
```
#Extract odd numbers from an array
arr_odd =arr[arr%2!=0]
arr_odd
```

Out[5]:
```
array([ 3,  5,  7,  9, 11])
```

In [6]:
```
#Replace all odd numbers with -1
arr[arr%2!=0]=-1
arr
```

Out[6]:
```
array([[-1,  4, -1],
       [ 6, -1,  8],
       [-1, 10, -1]])
```

In [7]:
```
#Get indices of elements greater than 5 [hint: check where method of numpy]
index=np.where(arr>5)
index
```

Out[7]:
```
(array([1, 1, 2]), array([0, 2, 1]))
```

In [8]:
```
#argwhere
index=np.argwhere(arr>5)
index
```

Out[8]:
```
array([[1, 0],
       [1, 2],
       [2, 1]])
```

In [9]:
```
#Access last row - last column
arr[-1:,-1:]
```

Out[9]:
```
array([[-1]])
```

In [10]:
```
#Set2
#Normalize a NumPy array [ to normalize means scaling the values of an array into a

A = np.array([10, 20, 30, 40, 50])
normalized = (A - A.min()) / (A.max() - A.min())

print("Original Array:")
print(A)
print("Normalized Array:")
print(normalized)
```

```
Original Array:
[10 20 30 40 50]
Normalized Array:
[0.   0.25 0.5  0.75 1.  ]
```

In [11]:
```
#Create a 5×5 array with 1 on border and 0 inside
arr1 = np.full(25,1).reshape(5,5)
arr1
```

Out[11]:
```
array([[1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1]])
```

In [12]:
```
arr1[1:-1,1:-1]=0
arr1
```

Out[12]:
```
array([[1, 1, 1, 1, 1],
       [1, 0, 0, 0, 1],
       [1, 0, 0, 0, 1],
       [1, 0, 0, 0, 1],
       [1, 1, 1, 1, 1]])
```

In [13]:
```
#Create a random array of 10×10: perform stats (min, max, mean, std)
arr3 =np.random.randint(200,size=(10,10))
arr3
```

Out[13]:
```
array([[135, 177, 130,  99,  60,  82,  85, 192,   3,  17],
       [ 50, 110, 168, 167, 122, 198,  44, 199, 191, 179],
       [ 87, 181,   1,  66,  40, 134, 123, 103,  83,   7],
       [196, 120,  97, 150, 115,  52,  35,   7,  94,  34],
       [ 95,  20,  56, 132, 161,  18,  19, 134, 119, 121],
       [ 88,  60, 146,  36, 116,  55, 135,  57,  48, 134],
       [ 19,  51, 165, 120, 142, 116, 129, 169, 102, 168],
       [ 82, 159,  30, 197,  81, 167, 188,  96,  75, 109],
       [ 52, 130,  77,  12, 156,  38, 179,  35,   0, 152],
       [ 50, 181, 171,   5,  93, 135,   5, 110, 117, 150]], dtype=int32)
```

In [14]:
```
arr3.min()
```

Out[14]:
```
np.int32(0)
```

In [15]:
```
arr3.max()
```

Out[15]:
```
np.int32(199)
```

In [16]:
```
np.mean(arr3)
```

Out[16]:
```
np.float64(100.96)
```

In [17]:
```
np.std(arr)
```

Out[17]:
```
np.float64(4.245549594342845)
```

In [18]:
```
#Broadcast a 1D array over a 2D array [all possible scenarios]
arr4=np.array([[9,77,10],
               [56,8,9],
               [55,80,2]])
arr4
```

Out[18]:
```
array([[ 9, 77, 10],
       [56,  8,  9],
       [55, 80,  2]])
```

In [19]:
```
arr5 =np.array([20,10,3])
arr5
```

Out[19]:
```
array([20, 10,  3])
```

In [20]:
```
#Addition
arr4+arr5
```

Out[20]:
```
array([[29, 87, 13],
       [76, 18, 12],
       [75, 90,  5]])
```

In [21]:
```
#Subtraction
arr4-arr5
```

Out[21]:
```
array([[-11,  67,   7],
       [ 36,  -2,   6],
       [ 35,  70,  -1]])
```

In [22]:
```
#MUltiplication
arr4*arr5
```

Out[22]:
```
array([[ 180,  770,   30],
       [1120,   80,   27],
       [1100,  800,    6]])
```

In [23]:
```
#Division
arr4//arr5
```

Out[23]:
```
array([[0, 7, 3],
       [2, 0, 3],
       [2, 8, 0]])
```

In [24]:
```
#Modulus
arr4%arr5
```

Out[24]:
```
array([[ 9,  7,  1],
       [16,  8,  0],
       [15,  0,  2]])
```

In [25]:
```
#Sort a 2D array by column 1 [hint: check method argsort()]
A = np.array([[5, 2, 8],
              [1, 9, 3],
              [4, 6, 7],
              [2, 1, 5]])

print("Original Array:")
print(A)
sorted_array = A[A[:,1].argsort()]

print("Sorted Array by Column 1:")
print(sorted_array)
```

```
Original Array:
[[5 2 8]
 [1 9 3]
 [4 6 7]
 [2 1 5]]
Sorted Array by Column 1:
[[2 1 5]
 [5 2 8]
 [4 6 7]
 [1 9 3]]
```

In [26]:
```
#Set3
#Find common elements between two arrays [with and without using intersect1d, use i
A = np.array([1,2,3,4,5])
B = np.array([3,4,5,6,7])

common = np.intersect1d(A,B)
print("Common elements:", common)
```

```
Common elements: [3 4 5]
```

In [27]:
```
#Find common elements between two arrays [with and without using intersect1d, use i
A = np.array([1,2,3,4,5,10])
B = np.array([3,4,5,6,7])

common = A[np.isin(A,B)]
print("Common elements:", common)
```

```
Common elements: [3 4 5]
```

In [28]:
```
#Subtract row mean from each row of a 2D array
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
row_mean = A.mean(axis=1, keepdims=True)
result = A - row_mean
```

```
print("Original Array:")
print(A)
print("Row Means:")
print(row_mean)
print("Result:")
print(result)
```

```
Original Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Row Means:
[[2.]
 [5.]
 [8.]]
Result:
[[-1.  0.  1.]
 [-1.  0.  1.]
 [-1.  0.  1.]]
```

In [29]:
```
#Given a 4×4 array, swap the first half rows with the second half
A = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9,10,11,12],
              [13,14,15,16]])

print("Original Array:")
print(A)
mid=len(A)//2
# Swap first half rows with second half rows
result = np.vstack((A[mid:], A[:mid]))

print("After Swapping Rows:")
print(result)
```

```
Original Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
After Swapping Rows:
[[ 9 10 11 12]
 [13 14 15 16]
 [ 1  2  3  4]
 [ 5  6  7  8]]
```

In [30]:
```
#Given a 1D array of random numbers, find the indices of the 5 largest values (no s
# Create a 1D array of random numbers
A = np.random.randint(1,100,10)

print("Array:")
print(A)

# Find indices of 5 largest values
indices = A.argsort()[-5:]

print("Indices of 5 largest values:")
```

```
print(indices)

print("5 largest values:")
print(A[indices])
```

```
Array:
[60  5 80 91 24 56 73 66 19 71]
Indices of 5 largest values:
[7 9 6 2 3]
5 largest values:
[66 71 73 80 91]
```

In [31]:
```
#From a 2D NumPy array, extract only the unique rows (no duplicates hint: check uni
A = np.array([[1,2,3],
              [4,5,6],
              [1,2,3],
              [7,8,9],
              [4,5,6]])

print("Original Array:")
print(A)

# Extract unique rows
unique_rows = np.unique(A, axis=0)

print("Unique Rows:")
print(unique_rows)
```

```
Original Array:
[[1 2 3]
 [4 5 6]
 [1 2 3]
 [7 8 9]
 [4 5 6]]
Unique Rows:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

In [ ]:
```

```